

Informe Técnico: Planificación de trayectoria profesional

Adriana Boué García C311

Junio 2026

1 Descripción del Problema

El presente proyecto aborda el problema de **Planificación de Trayectorias Profesionales**. El objetivo fundamental consiste en encontrar una secuencia ordenada de acciones formativas (un plan de cursos) que permita a un usuario transitar desde un estado de conocimiento actual hasta alcanzar un estado deseado que contenga un conjunto de competencias o habilidades profesionales objetivo.

El sistema se enfrenta a un entorno complejo que va más allá de la planificación clásica tradicional, integrando restricciones de recursos acumulativos y variables de perfil. Para que un plan sea factible, debe satisfacer de forma simultánea cuatro dimensiones críticas:

1. **Dependencias Pedagógicas:** Cada curso posee un conjunto de prerrequisitos lógicos. Una acción formativa solo puede ejecutarse si el estudiante ha adquirido previamente dichas competencias.
2. **Limitación de Recursos Financieros:** La trayectoria total acumulada posee un costo monetario que no debe exceder el presupuesto máximo disponible por el usuario.
3. **Limitación de Recursos Temporales:** El tiempo total invertido en horas de instrucción académica debe ser menor o igual al plazo temporal del que dispone el usuario.
4. **Filtros de Idoneidad de Perfil:** Las acciones seleccionadas deben restringirse estrictamente a las preferencias cualitativas del usuario en cuanto a la modalidad de estudio (online, presencial, mixto) y el nivel de dificultad académica (baja, media, alta).

Dada la naturaleza de la entrada del problema (peticiones en lenguaje natural), el sistema incorpora un Modelo de Lenguaje (LLM) como componente funcional crítico en múltiples etapas. Por un lado, el LLM actúa en la fase inicial como un intérprete de objetivos en lenguaje natural y extractor de entidades encargado de estructurar las condiciones de inicio, las competencias de la

meta y los límites de recursos. Por otro lado, interviene en la etapa de salida operando como un evaluador cualitativo de las trayectorias propuestas, analizando la viabilidad pedagógica y la coherencia del camino generado en relación con las aspiraciones iniciales del usuario.

2 Modelado Formal del Problema

Basándonos en la formalización clásica de una instancia de STRIPS del curso, modelamos el sistema adaptando el concepto de la tupla original para un espacio de estados mixto (proposicional y numérico) que refleja fielmente la lógica programada en el planificador.

Formalmente, el problema se define como la tupla:

$$\mathcal{P} = \langle P, O, I, G, R_{extra} \rangle$$

Donde cada uno de los componentes se describe algebraicamente a continuación:

2.1 Conjunto de Condiciones y Espacio de Estados

Sea $P = \{p_1, p_2, \dots, p_m\}$ el conjunto finito de todas las condiciones (variables proposicionales). Cada condición $p \in P$ representa una habilidad académica del sistema.

A diferencia del STRIPS básico, un **estado** s en nuestro sistema es un par ordenado que acopla las variables lógicas y los recursos numéricos disponibles en ese instante:

$$s = \langle S_{skills}, S_{resources} \rangle$$

Donde:

- $S_{skills} \subseteq P$: Conjunto de condiciones (habilidades) que son verdaderas en dicho estado (bajo la hipótesis de mundo cerrado).
- $S_{resources} = \{\text{money} : v_m, \text{time} : v_t\}$ donde $v_m, v_t \in R^+$ representan los balances remanentes de dinero y tiempo.

2.2 Conjunto de Operadores (O)

Sea O el conjunto de operadores disponibles (cursos). Cada operador $o \in O$ se define formalmente a través de la siguiente estructura:

$$o = \langle \text{id}_o, \text{mod}_o, \text{diff}_o, \alpha_o, \beta_o, \gamma_o, \delta_o \rangle$$

Donde cada componente se corresponde directamente con la implementación del código:

- id_o : Identificador único del curso.

- $\text{mod}_o \in \{\text{online}, \text{presencial}, \text{mixto}\}$: Modalidad del curso.
- $\text{diff}_o \in \{\text{baja}, \text{media}, \text{alta}\}$: Nivel de dificultad pedagógica.
- $\alpha_o \subseteq P$: Precondiciones positivas; conjunto de habilidades que deben ser verdaderas para habilitar el curso.
- $\beta_o \subseteq P$: Precondiciones negativas; conjunto de habilidades que deben ser falsas. En este modelo, $\beta_o = \emptyset$.
- $\gamma_o \subseteq P$: Lista de adición (“Add list”); condiciones que se hacen verdaderas tras completar la acción (efectos/habilidades adquiridas).
- δ_o : Lista de eliminación (“Delete list”). En este modelo, δ_o no elimina variables proposicionales de conocimiento, sino que representa el consumo cuantificable de recursos del sistema, definido como el par numérico:

$$\delta_o = \{\text{money} : \text{cost}_o, \text{time} : \text{duration}_o\}$$

2.3 Estado Inicial (I)

El estado inicial I define el punto de partida del espacio de búsqueda a partir de los datos provistos por el usuario:

$$I = \langle \text{start_skills}, \text{resources}_0 \rangle$$

Donde $\text{resources}_0 = \{\text{money} : \text{money_requested}, \text{time} : \text{time_requested}\}$.

2.4 Especificación del Estado Meta (G)

La meta está dada por la condición de parada del planificador. Un estado $s = \langle S_{\text{skills}}, S_{\text{resources}} \rangle$ califica como meta si el conjunto de habilidades objetivo del usuario está completamente contenido en las habilidades del estado actual:

$$G = \{s \in \text{Espacio de Estados} \mid \text{goal_skills} \subseteq S_{\text{skills}}\}$$

2.5 Condición de Aplicabilidad de un Operador

Un operador $o \in O$ es legalmente aplicable en un estado actual $s = \langle S_{\text{skills}}, S_{\text{resources}} \rangle$ si y solo si se cumplen de manera simultánea todas las condiciones lógicas y de recursos definidas en la función `is_applicable`:

$$\text{Aplicable}(o, s) \iff \begin{cases} \alpha_o \subseteq S_{\text{skills}} & (\text{Prerrequisitos cumplidos}) \\ S_{\text{skills}} \cap \beta_o = \emptyset & (\text{Precondiciones neg. vacías}) \\ \gamma_o \not\subseteq S_{\text{skills}} & (\text{Aporta habilidades nuevas}) \\ \text{mod}_o \in \text{modalities_requested} & (\text{Filtro de modalidad}) \\ \text{diff}_o \in \text{difficulties_requested} & (\text{Filtro de dificultad}) \\ \delta_o[\text{'money'}] \leq S_{\text{resources}}[\text{'money'}] & (\text{Presupuesto suficiente}) \\ \delta_o[\text{'time'}] \leq S_{\text{resources}}[\text{'time'}] & (\text{Tiempo suficiente}) \end{cases}$$

2.6 Función de Transición (Función Sucesor)

La función de transición mapea el estado actual y el operador hacia un nuevo estado sucesor calculando la unión de competencias lógicas y la sustracción de los recursos consumidos indicados en la lista de eliminación δ_o :

$$\text{sucesor} : \text{Estado} \times O \rightarrow \text{Estado}$$

$$\text{sucesor}(s, o) = \langle S_{skills} \cup \gamma_o, S'_{resources} \rangle$$

Donde el nuevo balance de recursos $S'_{resources}$ se computa sustrayendo los valores indexados en δ_o :

$$S'_{resources}[\text{'money'}] = S_{resources}[\text{'money'}] - \delta_o[\text{'money'}]$$

$$S'_{resources}[\text{'time'}] = S_{resources}[\text{'time'}] - \delta_o[\text{'time'}]$$

3 Descripción del Dataset Utilizado y Proceso de Generación

Para la validación empírica y la evaluación del rendimiento de los algoritmos de búsqueda en espacios de estados (BFS, UCS, A^*) junto al componente de planificación adaptativa, se diseñó y estructuró un conjunto de datos sintético curado en formato JSON que modela el dominio académico de las *Ciencias de la Computación*.

3.1 Justificación de la Generación Sintética frente a Datos Reales

La naturaleza del problema conceptualizado bajo el paradigma de planificación formal STRIPS exige que cada curso actúe estrictamente como un operador lógico. Esto requiere que cada instancia posea precondiciones declarativas explícitas (α) y una lista de adición de efectos (γ), vinculados a variables cuantitativas de consumo de recursos de coste financiero y temporal (δ).

Los datasets públicos disponibles en plataformas de datos abiertos (procedentes de plataformas comerciales como Udemy o Coursera) contienen metadatos de marketing como reseñas, número de inscritos o precios históricos, pero carecen por completo de un etiquetado formal de dependencias académicas y habilidades discretas interconectadas. Utilizar datos crudos reales habría obligado a diseñar un sistema intermedio de extracción heurística propenso a introducir ruido estructural (tales como grafos inconexos, callejones sin salida o bucles lógicos infinitos), lo cual desviaría el foco central de la investigación: evaluar con precisión la eficiencia, la completitud y la optimalidad de los algoritmos de búsqueda artificial.

3.2 Estructura General del Dataset

El dataset se compone de tres bloques fundamentales de información que mapean directamente las abstracciones de nuestro modelo formal:

1. **Metadatos de Dominio:** Atributos cualitativos que definen el contexto conceptual (*domain*) y el tema de planificación asignado.
2. **Universo de Habilidades (available_skills):** Un catálogo cerrado compuesto por un conjunto finito de 30 competencias clave (P) indexadas en el mercado del software actual (ej. *programacion_basica*, *estructuras_datos*, *machine_learning*). Estas variables lógicas configuran los componentes internos de los estados y definen los perfiles de entrada y salida académica.
3. **Catálogo de Operadores (courses):** Un listado de cursos estructurados como objetos lógicos. Cada curso representa un operador ($o \in O$) cuyas propiedades cuantitativas y cualitativas de consumo se sintetizan en la Tabla 1.

Table 1: Métricas y rangos de diseño de los operadores del dataset.

Atributo del Operador	Tipo de Datos	Rango / Valores Disponibles
Identificador (<i>id</i>)	Alfanumérico	Cadena única por curso
Precondiciones (α)	Conjunto $\subseteq P$	Desde $\emptyset$ hasta 2 habilidades previas
Lista de Adición (γ)	Conjunto $\subseteq P$	Desde 1 hasta 3 habilidades adquiridas
Consumo de Dinero (δ_{money})	R^+	\$0 (Cursos gratuitos) hasta \$650
Consumo de Tiempo (δ_{time})	R^+	10 horas hasta 180 horas
Modalidad (<i>modality</i>)	Categorico	{online, presencial, mixto}
Dificultad (<i>difficulty</i>)	Categorico	{baja, media, alta}

3.3 Proceso de Generación y Curación

El proceso de construcción del dataset se dividió en tres fases metodológicas:

1. **Diseño del Dominio Lógico (Estructura de Red):** Se distribuyeron las habilidades en niveles de profundidad pedagógica. Cursos iniciales como `fundamentos_programacion_python` no poseen prerequisites ($\alpha = \emptyset$), actuando como los nodos raíz que permiten iniciar la propagación en estados vacíos ($I = \emptyset$). Cursos avanzados como `algoritmos_avanzados_academia` exigen competencias previas, forzando al planificador a realizar encadenamientos hacia adelante.
2. **Generación Asistida por LLM:** Utilizando prompts fuertemente estructurados con restricciones de formato (*JSON schema*), se instruyó a un modelo de lenguaje avanzado para poblar el catálogo de operadores.

El modelo aportó semántica de alta calidad del mundo real (títulos realistas, descripciones fluidas y coherentes) mientras aseguraba que las listas de precondiciones y efectos respetaran una jerarquía académica natural.

3. **Auditoría de Grafos y Ajuste Manual:** Se realizó una revisión analítica manual de la estructura de la red para asegurar que el Grafo Dirigido Acíclico (DAG) resultante fuera completamente resoluble, verificando la inexistencia de dependencias circulares (ej. que un curso A requiriera un curso B y viceversa). Durante esta fase, se crearon y modificaron diversos cursos de manera manual con el fin de equilibrar los costes y tiempos, inyectando redundancia estratégica para forzar rutas competitivas: programas tipo *Bootcamp* (cortos en tiempo pero costosos monetariamente) frente a programas tipo *MOOC* (largos y extensos pero económicos o gratuitos).

3.4 Justificación de Idoneidad frente al Problema Definido

El dataset cumple estrictamente con las condiciones matemáticas y operativas del problema de planificación por las siguientes razones:

- **Evaluación de Costes (UCS):** Al incluir cursos gratuitos ($\delta_{\text{money}} = 0$) frente a ofertas de pago, el algoritmo de Búsqueda de Costo Uniforme (*Uniform Cost Search*) puede demostrar su optimalidad matemática al encontrar siempre la trayectoria que minimice los recursos financieros consumidos.
- **Admisibilidad de la Heurística (A^*):** La asignación limpia y consistente de las habilidades obtenidas (γ) permite que la función heurística implementada en el sistema (basada en la diferencia de conjuntos entre el estado actual y las metas finales) sea admisible ($h(n) \leq h^*(n)$) y consistente. Esto garantiza teóricamente que el algoritmo A^* devuelva la solución óptima expandiendo significativamente menos nodos que BFS y UCS.
- **Disparador del Planificador Adaptativo:** La distribución balanceada de modalidades e intervalos de presupuesto proporciona el entorno perfecto para que, ante peticiones de usuarios con restricciones extremas e incompatibles (ej. “Aprender Machine Learning en 10 horas con 0 dólares”), el motor determinista falle de manera controlada (`is_applicable` retorna falso para todos los caminos métricos), gatillando con éxito el módulo adaptativo soportado cualitativamente por el LLM para relajar las restricciones del usuario.

4 Diseño del Algoritmo

El núcleo algorítmico del sistema implementa un motor de búsqueda gráfica que opera sobre el espacio de estados mixto definido en la formalización clásica.

Para resolver el problema de planificación, se han diseñado e implementado tres variantes estratégicas de búsqueda en grafos de estados (BFS, UCS y A^*), complementadas por un mecanismo superior de envoltura denominado Planificador Adaptativo. Este último se encarga de gestionar la resiliencia del sistema ante la ausencia de soluciones factibles en el espacio determinista estricto.

4.1 Estrategias de Búsqueda No Informada y de Costo Uniforme

4.1.1 Búsqueda en Anchura (BFS)

El algoritmo BFS explora el espacio de estados de manera uniforme por niveles de profundidad. Utiliza una estructura de datos de cola FIFO (*First-In, First-Out*) para garantizar que la primera trayectoria que satisfaga la condición de parada $\text{goal.skills} \subseteq S_{\text{skills}}$ sea aquella que posea el menor número absoluto de operadores (cursos), independientemente del costo financiero o la duración horaria de estos. Su función de control evita ciclos redundantes mediante el almacenamiento de las firmas de los estados de conocimiento dominados (**frozenset**) en un conjunto histórico de cerrado (*visited set*).

4.1.2 Búsqueda de Costo Uniforme (UCS)

Para los escenarios donde el usuario prioriza la economía de la ruta, el planificador implementa UCS utilizando una cola de prioridad estructurada como un montículo mínimo (*Min-Heap*). A diferencia de BFS, UCS extrae en cada iteración el nodo que minimiza la función de costo acumulado real del camino $g(n)$.

Dado que el dominio gestiona recursos heterogéneos (moneda y tiempo continuos), se diseñó una Función de Costo Escalada y Ponderada para el paso de un operador. Para evitar que las magnitudes numéricas nativas de los precios distorsionen la toma de decisiones frente a las horas de dedicación, los consumos de cada operador se normalizan con respecto a las cotas máximas iniciales previstas por el usuario ($M_{\text{max}}, T_{\text{max}}$):

$$g(o) = w_m \cdot \left(\frac{\delta_o[\text{'money'}]}{M_{\text{max}}} \right) + w_t \cdot \left(\frac{\delta_o[\text{'time'}]}{T_{\text{max}}} \right)$$

Donde $w_m, w_t \in [0, 1]$ representan los factores de ponderación lineal introducidos por el usuario (cumpliendo $w_m + w_t = 1$). El costo acumulado final del camino es la suma de los costos escalados de sus operadores: $g(n) = \sum g(o_i)$. UCS garantiza optimalidad matemática respecto a esta métrica compuesta.

4.2 Búsqueda Informada (A^*) y Diseño de la Heurística

El algoritmo A^* optimiza la exploración combinando el costo real acumulado $g(n)$ con un mecanismo de estimación prospectiva dada por una función heurística

$h(n)$. La función de evaluación que gobierna el orden de extracción en el montículo de prioridad responde a la ecuación clásica:

$$f(n) = g(n) + w_c \cdot h(n)$$

Donde w_c representa el peso asignado al costo estructural del camino de cursos.

4.2.1 Modelado de la Función Heurística

Para guiar el planificador de forma efectiva a través del grafo de prerrequisitos, se diseñó una función heurística no lineal basada en la diferencia de conjuntos del estado de conocimiento. Sea S_{skills} el conjunto de habilidades dominadas en el nodo actual y G_{skills} el conjunto de habilidades meta. Definimos el conjunto de competencias faltantes como:

$$\mathcal{M}(n) = G_{skills} \setminus S_{skills}$$

Si $\mathcal{M}(n) = \emptyset$, el nodo es un estado meta y $h(n) = 0$. En caso contrario, se define el subconjunto de operadores útiles del dataset que aportan directamente al menos una competencia de la meta como:

$$\mathcal{O}_{useful} = \{o \in O \mid \gamma_o \cap \mathcal{M}(n) \neq \emptyset\}$$

Si $\mathcal{O}_{useful} = \emptyset$, el estado actual es un callejón sin salida del cual es imposible alcanzar el objetivo con los cursos existentes, asignando analíticamente un costo $h(n) = \infty$. Si existen cursos viables, se calcula el máximo grado de solapamiento de un único operador frente a las carencias del usuario:

$$\mu_{\max} = \max_{o \in \mathcal{O}_{useful}} |\gamma_o \cap \mathcal{M}(n)|$$

La función heurística final computa una penalización cuadrática proporcional a la cantidad de conceptos faltantes, atenuada por el potencial de cobertura instantánea del mejor operador del entorno:

$$h(n) = \frac{|\mathcal{M}(n)|^2}{\max(1, \mu_{\max})}$$

4.2.2 Discusión de Admisibilidad y Consistencia

La heurística diseñada es admisible ($h(n) \leq h^*(n)$) en entornos con costos unitarios puros, ya que el término cuadrático penaliza el volumen de carencias lógicas remanentes, mientras que el denominador estima el salto máximo de absorción de conocimientos en un solo paso. Al ser una función monótona en la tasa de reducción de diferencias del grafo dirigido de habilidades, se comporta de forma consistente, evitando la re-evaluación de nodos en el conjunto cerrado y acelerando exponencialmente la convergencia de A^* en comparación con UCS y BFS.

4.3 Planificación Adaptativa y Espacios de Relajación

Una limitación crítica de la planificación clásica bajo STRIPS es su fragilidad ante restricciones hiper-acotadas: si un usuario solicita una meta ambiciosa con límites financieros o temporales severos, la función `is_applicable` bloqueará todas las transiciones y el motor determinista fallará retornando un plan vacío. Para resolver esta vulnerabilidad, se diseñó e implementó un Planificador Adaptativo que opera sobre una arquitectura de evaluación cualitativo-cuantitativa.

4.3.1 Algoritmo de Mutación de Restricciones

Cuando el motor estricto de A^* falla, el planificador adaptativo extrae el perfil de restricciones activas del usuario (`get_user_constraints_profile`) e induce un espacio de búsqueda alternativo mediante la creación de un conjunto finito de Candidatos de Relajación. Cada candidato representa una mutación aislada o combinada del espacio físico y cualitativo del problema:

- **Mutación Financiera:** Incrementa de forma aislada el presupuesto financiero en un 30% ($M_{mutado} = 1.30 \cdot M_0$).
- **Mutación Temporal:** Incrementa la holgura del tiempo disponible en un 30% ($T_{mutado} = 1.30 \cdot T_0$).
- **Mutación de Catálogo (Modalidad):** Abre el espectro discretizado forzando la inclusión de todas las modalidades del sistema ($\{\text{online, presencial, mixto}\}$).
- **Mutación de Catálogo (Dificultad):** Libera las barreras académicas permitiendo cualquier nivel de complejidad pedagógica ($\{\text{baja, media, alta}\}$).
- **Relajación Agregada:** Aplica simultáneamente todas las holguras anteriores en un escenario de margen amplio.

4.3.2 Métrica Cualitativa del Sacrificio del Usuario

Cada escenario candidato modula los parámetros del planificador y re-ejecuta internamente el algoritmo A^* . Para seleccionar de manera óptima el plan adaptativo que implique el menor impacto negativo sobre las preferencias originales del usuario, se formuló una función matemática cuantitativa denominada Puntaje de Relajación ($\mathcal{S}_R$):

$$\mathcal{S}_R = C_{modality} + C_{difficulty} + \Phi(M_{mutado}, M_0) + \Phi(T_{mutado}, T_0)$$

Donde el costo de romper un filtro discreto de modalidad o dificultad es penalizado con un valor plano:

$$C_{filtro} = \begin{cases} 1.0 & \text{si el conjunto fue alterado} \\ 0.0 & \text{si se mantuvo idéntico} \end{cases}$$

Y la penalización para los recursos continuos de dinero y tiempo se evalúa mediante el desvío porcentual continuo respecto a la cota original del usuario:

$$\Phi(V_{mutado}, V_0) = \begin{cases} \frac{|V_{mutado} - V_0|}{V_0} & \text{si } V_0 > 0 \\ 1.0 & \text{si } V_0 = 0 \text{ y } V_{mutado} > 0 \end{cases}$$

El planificador adaptativo evalúa los subgrafos alternativos y selecciona la trayectoria académica asociada al mínimo puntaje de relajación ($\min \mathcal{S}_R$). En caso de que las relajaciones controladas fallen debido a incompatibilidades extremas del dataset, el sistema activa un Modo Forzado Crítico, rompiendo el 100% de los límites lógicos y numéricos del espacio de búsqueda para garantizar la devolución de una ruta didáctica viable. Esta acción levanta una bandera de advertencia (**forced_success**) diseñada para ser interpretada cualitativamente por el Modelo de Lenguaje en la etapa de interfaz de salida.

5 El Rol del LLM en el Sistema

El Modelo de Lenguaje de Gran Escala (*Large Language Model* o LLM) no opera como el motor de inferencia lógico o resolutor de planes del sistema, sino como una interfaz semántica bidireccional acoplada al motor determinista. Su integración responde a dos necesidades fundamentales de la arquitectura: el sub-sistema de Extracción e Interpretación de Entidades Semánticas (traducción de lenguaje natural a las estructuras formales del modelo STRIPS) y el sub-sistema de Auditoría Cualitativa Cruzada (evaluación discursiva del espacio de soluciones e interpretación de la resiliencia adaptativa).

5.1 Sub-sistema de Extracción e Interpretación Semántica de Entrada

Para que el planificador pueda procesar una consulta en lenguaje natural U_{text} , el sistema debe mapear dicha cadena a los conjuntos simbólicos iniciales del problema: habilidades iniciales (I), metas finales (G), filtros de catálogo discretos ($C_{modality}, C_{difficulty}$) y cotas numéricas continuas de recursos (M_{max}, T_{max}). El sistema utiliza llamadas atómicas al LLM parametrizadas mediante *prompt engineering* estricto y tipado estructurado en formato JSON.

5.1.1 Extracción Segmentada de Conocimientos y Metas

Para mitigar los fenómenos de atenuación de atención en contextos largos y prevenir sesgos de ambigüedad temporal, el sistema aísla la extracción de metas de la extracción de conocimientos actuales mediante dos funciones diferenciadas: `get_goals_from_text` y `get_start_skill`.

Ambas funciones inyectan la lista de habilidades permitidas obligatorias e imponen una Guía Lingüística de Verbos Estricta basada en análisis temporal discursivo:

- **Análisis de Tiempo Presente (Experiencia):** Expresiones sintácticas como "sé", "conozco", "domino" o "actualmente" se mapean como precondiciones del estado inicial (I).
- **Análisis de Tiempo Futuro (Deseos):** Expresiones sintácticas como "quiero", "me interesa", "busco" o "aprender" se clasifican como parte del vector de objetivos (G).

El sistema implementa una Regla de Priorización Cruzada por Conflicto Simbólico ante enunciados mixtos (ej. "ya conozco bases de datos pero quiero mejorar"). Si la meta entra en colisión con el conocimiento actual, el extractor de objetivos prioriza el contexto de experiencia y la excluye de G para evitar planes redundantes que enseñen habilidades ya dominadas. Por el contrario, el extractor de conocimientos (`get_start_skill`) prioriza el contexto de meta futura y la excluye de I para forzar al planificador a incluir el curso de perfeccionamiento en la secuencia del grafo.

5.1.2 Mapeo de Filtros Categóricos Discretos

Las restricciones cualitativas del catálogo se extraen mediante las funciones `get_modality` y `get_difficulty`. A diferencia de las habilidades, estas variables operan como operadores de filtrado sobre el espacio de estados y requieren el procesamiento de cuantificadores de exclusividad y negación lógica:

- **Modalidad de Estudio ($C_{modality}$):** El LLM mapea las preferencias a los tokens discretos permitidos (*online*, *presencial*, *mixto*). El sistema implementa una heurística secundaria basada en expresiones regulares para resolver ambigüedades de rechazo: si el usuario introduce una negación explícita (ej. "no quiero presencial"), el sistema deduce el conjunto complementario compatible de forma automática (*'online'*, *'mixto'*). Sin embargo, ante la presencia de cuantificadores de exclusividad estricta (ej. "solo online"), el sistema anula la expansión lógica y restringe el vector exclusivamente a dicha opción. Si no se detectan preferencias, se devuelve la bandera NINGUNA, lo que inhabilita el filtro en el catálogo.
- **Dificultad Académica ($C_{difficulty}$):** El sub-sistema clasifica la entrada bajo tres escenarios operacionales: *Mención Explícita* (mapeo directo al conjunto discreto, p. ej., "avanzado" $\rightarrow$ "alta"), *Negación Explícita* (exclusión del elemento rechazado y retorno de las alternativas viables) y *Ausencia de Restricción* (retorno del token NINGUNA para habilitar la búsqueda libre). El LLM ejecuta un criterio de Aislamiento de Autoevaluación: las expresiones asociadas al estado del usuario (ej. "soy principiante" o "tengo experiencia") se ignoran en esta fase por tratarse de descriptores de precondición inicial (I), evitando alterar erróneamente el filtro de dificultad intrínseca de las asignaturas candidatos.

5.1.3 Normalización y Parseo de Recursos Continuos

Las funciones de extracción financiera (`get_money`) y temporal (`get_time`) transforman magnitudes descritas de forma libre a variables matemáticas acotadas. En particular, la función `get_time` realiza un proceso de normalización dimensional basado en una tasa de conversión pedagógica fija, transformando plazos macro-educativos a horas de dedicación neta bajo la siguiente función de mapeo por unidad:

$$T_{\max} = \begin{cases} \text{cantidad} \cdot 30 \cdot 8 & \text{si unidad} = \text{'meses'} \\ \text{cantidad} \cdot 7 \cdot 8 & \text{si unidad} = \text{'semanas'} \\ \text{cantidad} \cdot 8 & \text{si unidad} = \text{'días'} \\ \text{cantidad} \cdot 365 \cdot 8 & \text{si unidad} = \text{'años'} \\ \text{cantidad} & \text{si unidad} = \text{'horas'} \end{cases}$$

Donde se modela analíticamente una jornada académica estándar de 8 horas diarias de actividad efectiva. Ante la ausencia de restricciones explícitas (indicada semánticamente por la respuesta patrón -1), el sistema intercepta el flujo y asigna un valor virtual infinito (10^{13}) que actúa como elemento neutro en las funciones de aplicabilidad física del planificador.

5.2 Sub-sistema de Auditoría Cualitativa y Evaluación de Salida

Una vez que el motor de búsqueda en grafos genera las trayectorias académicas óptimas, el sistema vuelve a acoplar el LLM para construir la capa discursiva de salida. El modelo actúa como un auditor curricular ciego de nivel doctoral a través de dos metodologías avanzadas.

5.2.1 Preprocesamiento Numérico de Verdad Absoluta

Para neutralizar de forma absoluta la tendencia a la alucinación distributiva o aritmética del LLM al comparar planes, la función `compare_and_evaluate_trajectories` ejecuta un preprocesamiento algorítmico determinista. Este bloque calcula previamente los extremos numéricos e inyecta en el prompt un vector inmutable denominado Verdades Matemáticas Calculadas por el Sistema:

$$\mathcal{V} = \langle \arg \min g(n), \arg \max g(n), \arg \min T(n), \arg \max T(n), \arg \min |path|, \arg \max |path| \rangle$$

El prompt restringe explícitamente el espacio del discurso del LLM, prohibiendo cualquier contradicción relacional o factual con respecto a $\mathcal{V}$.

5.2.2 Auditoría Sectorial y Cruces de Proporción

Bajo esta restricción analítica, el LLM ejecuta una evaluación cualitativa de tres dimensiones sobre los planes resultantes:

1. **Eficiencia de la Proporción Costo-Beneficio ($\Delta g/\Delta T$):** Evalúa el *trade-off* de los operadores, identificando qué planes mitigan la fricción financiera sacrificando tiempo o viceversa.
2. **Densidad Horaria Estructural:** Contrasta si una ruta corta en número de pasos presenta el riesgo pedagógico de "cuellos de botella" cognitivos (pocas asignaturas pero sumamente masivas y densas), o si una ruta larga ofrece una trayectoria formativa ágil y fragmentada en micro-cápsulas de aprendizaje.
3. **Continuidad de la Curva Pedagógica:** El LLM analiza la secuencia inyectada de las variables de dificultad de los cursos en el orden temporal del plan. Si detecta transiciones que omiten los escalones intermedios (ej. Baja $\rightarrow$ Alta), el modelo introduce una advertencia cualitativa tipificada formalmente como un "*Salto abrupto y crítico de dificultad (Brecha pedagógica)*"; por el contrario, si la secuencia es monótona indexada (Baja $\rightarrow$ Media $\rightarrow$ Alta), la valida discursivamente como una "*Curva de aprendizaje balanceada y progresiva*".

Finalmente, ante escenarios resueltos por el planificador adaptativo, la función **evaluate adaptive trajectory** obliga al LLM a contrastar cualitativamente las desviaciones físicas sufridas en el subgrafo respecto al texto original de la consulta, justificando de forma empírica ante el usuario qué sacrificios de su preferencia original (incremento del coste o apertura de modalidades) fueron estrictamente necesarios para desbloquear la viabilidad didáctica del objetivo.

6 Metodología Experimental

Para validar empíricamente el sistema de planificación educativa distribuido en este trabajo, se diseñó un marco de pruebas integral que evalúa de forma aislada e integrada los dos sub-sistemas principales: la interfaz de extracción basada en el Modelo de Lenguaje Grande (*LLMInterface*) y el motor algorítmico de generación de trayectorias (*Planner*).

El entorno experimental se sustenta sobre un catálogo académico formalizado en un archivo estructurado `dataset.json`, el cual contiene la lista maestra de habilidades permitidas (S_{master}) y un conjunto balanceado de asignaturas caracterizadas individualmente por sus vectores dinámicos de costo monetario, duración en horas efectivas, modalidad de impartición y nivel de dificultad académica.

6.1 Variables del Sistema y Métricas de Rendimiento

El diseño de la evaluación clasifica los elementos operacionales bajo el siguiente esquema metodológico general:

- **Variables Independientes:** El perfil de entrada del usuario suministrado en lenguaje natural (variando el volumen de restricciones explícitas

de presupuesto, tiempo, modalidad y nivel de complejidad) y la variante algorítmica seleccionada para la resolución del grafo de estados (*BFS*, *UCS*, *A** o *Adaptive Mode*).

- **Variables Dependientes:** Las métricas de eficiencia cuantitativa del motor de búsqueda (número de *Nodos Expandidos* y *Tiempo de Ejecución de CPU* medido en milisegundos), la validez dimensional de la trayectoria (longitud medida en cantidad de cursos, costo financiero indexado en dólares y volumen de horas netas de dedicación) y el estado final de convergencia lógica de la consulta (*Status*: *OK*, *Sin Solución* o *Relajado OK*).
- **Variables de Control:** El uso de una base de conocimiento homogénea idéntica para todas las iteraciones simultáneas y la fijación de la función heurística admisible y consistente basada en la distancia de faltantes de habilidades:

$$h(n) = |G \setminus \text{effects}(S_n)|$$

6.2 Bancos de Pruebas y Protocolo de Ejecución

La infraestructura experimental se implementó mediante tres scripts de control en Python, diseñados para someter al sistema a evaluaciones atómicas, algorítmicas y de extremo a extremo (*end-to-end*):

6.2.1 Pruebas de Extracción Semántica Aislada (`test_llm_interface.py`)

Este componente evalúa la precisión cualitativa de la interfaz de lenguaje natural utilizando un banco de 13 cadenas complejas de usuario (*Examples #1 a #13*). El script aísla el comportamiento de `LLMInterface` obligándola a procesar estructuras sintácticas variadas (ej. negaciones explícitas como *"no me gustan los cursos mixtos"* o expresiones de experiencia previa). Su objetivo es verificar que los métodos de extracción transformen el texto libre de forma correcta en los vectores discretos y continuos requeridos por el planificador, aplicando retardos controlados de 0.5 segundos entre llamadas para respetar las tasas de cuota del modelo subyacente.

6.2.2 Pruebas del Motor de Búsqueda Puro (`test_planner.py`)

Diseñado para auditar el rendimiento matemático de los algoritmos de resolución sobre un catálogo estático. Este script unifica 14 casos de prueba distribuidos en dos categorías operacionales:

- **Grupo 1: Evaluación de Optimalidad y Espacio de Estados (*Test Cases Easy*):** Compuesto por 10 escenarios base (*Test1* a *Test10*) donde las restricciones de tiempo ($T_{\max}$) y dinero ($M_{\max}$) permiten la convergencia teórica de los enfoques estrictos. Contrasta la eficiencia computacional de *A** contra la búsqueda ciega de *BFS* y la exploración por costo de *UCS*

bajo metas complejas como desarrollo web o flujos de ingeniería avanzados (*DevOps Pro*).

- **Grupo 2: Evaluación de Resiliencia y Flexibilidad Lógica (*Test Cases Hard*):** Diseñado con 4 escenarios de alta fricción matemática (*Test11* a *Test14*) creados para forzar fallos por sobre-restricción en los algoritmos tradicionales. Configura contradicciones físicas insalvables (ej. criptografía en un plazo crítico de 2 días o redes de computadoras con un límite de \$30 USD) para evaluar exclusivamente la activación del *Adaptive Planner* y sus rutinas de relajación en cascada.

6.2.3 Pruebas de Tubería Completa e Integración (*test_career_planner.py*)

Representa la validación del flujo completo de producción del sistema (*Career-Planner*). El script carga el entorno dinámico desde *dataset.json*, extrae las propiedades de modalidad y dificultad globales, e inicializa de manera secuencial 12 casos de uso reales en lenguaje natural.

El protocolo de ejecución para cada caso sigue un flujo lineal estricto: el texto es procesado por el LLM; si este identifica metas válidas, transfiere las variables normalizadas al constructor de la clase *Planner*. El sistema ejecuta concurrentemente las variantes estrictas (BFS, UCS y A^*) y almacena sus salidas. Si todas las variantes fallan debido a las restricciones, el pipeline activa de forma automática el *Adaptive Planner* como mecanismo de respaldo. Finalmente, todas las trayectorias descubiertas y unificadas son redirigidas al componente de auditoría cualitativa del LLM (*compare_and_evaluate_trajectories*) para emitir el veredicto y justificación pedagógica final al usuario.

7 Resultados y Análisis

En esta sección se exponen y discuten los hallazgos empíricos derivados de la ejecución de los tres bancos de pruebas configurados. El análisis se divide metodológicamente en la precisión del extractor semántico, la eficiencia computacional comparativa del motor de búsqueda y la resiliencia del pipeline integrado de extremo a extremo (*end-to-end*).

7.1 Evaluación de la Interfaz de Lenguaje Natural (LLMInterface)

La evaluación aislada a través de *test_llm_interface.py* se ejecutó sobre un banco de pruebas compuesto por 13 escenarios de entrada en lenguaje natural (*Examples #1* a *#13*). Esta suite fue diseñada intencionalmente para evaluar la tolerancia al ruido semántico, la detección de negaciones, la conversión de unidades temporales y el mapeo de competencias previas contra el catálogo estático de 30 habilidades del sistema.

Los resultados demuestran la robustez del modelo base *llama-3.3-70b-versatile* para estructurar la ambigüedad lingüística hacia el espacio de estados lógicos formalizado. Para ilustrar los mecanismos internos de extracción, se analizan a

continuación los fenómenos numéricos y semánticos más representativos observados en dicha suite:

1. **Filtrado de Metas Fuera de Dominio e Inyección de Centinelas (Caso #1):** Al procesar la consulta *"Quiero ser chef profesional"*, el componente demostró un comportamiento alineado al principio de seguridad defensiva del planificador. El LLM extrajo un vector de metas vacío (`'goals_extracted': []`) al no hallar correlación semántica con ninguna de las competencias de la base de conocimiento. Ante la ausencia de restricciones explícitas de presupuesto o plazos en el texto, el extractor inicializó los campos numéricos continuos (*money* y *time*) con un valor centinela infinito estandarizado en 10^{13} , evitando el colapso por variables nulas en el motor de búsqueda.

2. **Normalización Escalar y Extracción Discreta (Casos #2 al #4):** En el resto de la suite, el extractor mapeó con éxito variables implícitas complejas. En el Caso #2 (*"solo 3 meses"*), el sistema realizó una conversión matemática e inferencia temporal a horas netas equivalentes (720 horas), discriminando de forma exacta los filtros booleanos de modalidad (`['online']`). Por su parte, el Caso #3 demostró la capacidad de deducción de condiciones iniciales al identificar la habilidad `'bases_de_datos'` dentro del vector `start_skills_extracted`, permitiendo al planificador podar el grafo desde la raíz. Finalmente, el Caso #4 (*"desde cero"*) validó la correcta inicialización vacía del estado inicial frente a usuarios novatos, asignando la meta base de `'programacion_basica'`.

7.2 Análisis Comparativo del Motor de Búsqueda (Grupo 1: Casos Fáciles)

Los experimentos cronometrados sobre los 10 escenarios base del Grupo 1 arrojaron discrepancias críticas en términos de eficiencia computacional (*CPU time*) y uso de memoria (*nodos expandidos*). La Tabla 2 condensa las métricas clave de una muestra representativa de estos ensayos.

Table 2: Métricas de Rendimiento Computacional y Optimalidad (Grupo 1)

Caso de Prueba	Algoritmo	Status	Tiempo (ms)	Nodos Exp.	Long. (Cursos)
Test 1: JS y Dev Web ($M_{\max} = 500$, $T_{\max} = 240$)	BFS	OK	3.54	66	3
	UCS	OK	73.27	1358	3
	A*	OK	1.69	7	3
Test 2: Bases de Datos ($M_{\max} = 250$, $T_{\max} = 180$)	BFS	OK	6.08	98	3

Continúa en la siguiente página...

Caso de Prueba	Algoritmo	Status	Tiempo (ms)	Nodos Exp.	Long. (Cursos)
	UCS	OK	14.64	222	4
	A*	OK	4.26	21	3
Test 3: Front + Back ($M_{\max} = 300$, $T_{\max} = 250$)	BFS	OK	0.76	16	2
	UCS	OK	92.99	1980	3
	A*	OK	0.82	5	2
Test 9: Machine Learning ($M_{\max} = 5000$, $T_{\max} = 1200$)	BFS	OK	177.62	2198	5
	UCS	OK	556.21	8388	6
	A*	OK	142.43	1412	5
Test 10: IA Completa ($M_{\max} = 10^6$, $T_{\max} = 10^6$)	BFS	OK	120.49	1563	5
	UCS	OK	593.44	8423	6
	A*	OK	177.11	1729	5

7.2.1 Discusión Algorítmica de Eficiencia y Consecución de Objetivos

- **Comportamiento de BFS:** Al ser una búsqueda ciega por niveles, BFS minimiza de forma estricta la *longitud de la ruta* (número de aristas/cursos). En escenarios como el **Test 1** y el **Test 3**, converge con rapidez en tiempo de CPU (0.76 ms). No obstante, carece de noción de costo financiero, lo que se evidencia en el **Test 1**, donde escoge una ruta de \$100 dólares frente a la de \$95 hallada por UCS/A*, o en el **Test 2**, donde la trayectoria es significativamente más cara (\$230 vs \$10).
- **Explosión Combinatoria en UCS:** UCS garantiza optimalidad en costos financieros acumulados. Sin embargo, padece de una severa ineficiencia espacial y temporal. En el **Test 9** (Ruta de Deep Learning), expandió la alarmante cantidad de 8,388 nodos, requiriendo 556.21 ms para converger. Esto se debe a que explora de manera monótona todas las ramas de bajo costo relativo antes de progresar hacia estados que satisfacen los prerrequisitos avanzados.
- **Dominancia e Inferencia de A*:** El uso de la heurística admisible $h(n) = |G \setminus \text{effects}(S_n)|$ mitigó drásticamente el espacio de búsqueda. En el **Test 1**, A* alcanzó la misma solución óptima en costo de UCS (\$95), pero reduciendo los nodos expandidos de 1,358 a solo 7 nodos, con una aceleración computacional del 4335% (1.69 ms). Se demuestra así que la distancia orientada a metas rompe la simetría de la exploración ciega.

7.3 Resiliencia y Flexibilidad Lógica (Grupo 2: Casos Difíciles)

Los escenarios del Grupo 2 validaron de forma empírica la necesidad del *Adaptive Planner*. Ante entornos sobre-restringidos donde el conjunto de soluciones viables para los métodos tradicionales es vacío ($\emptyset$), el sistema activó con éxito las rutinas de relajación en cascada.

Como se observa en el resumen consolidado de la consola, los casos `test11`, `test12`, `test13` y `test14` mutaron su estado de **Sin Solución** hacia **Relajado OK** o **Forced OK**. Al no operar sobre un grafo estático homogéneo, el número de nodos expandidos reporta un estado adaptativo multi-grafo, garantizando que el usuario reciba una alternativa subóptima pero físicamente construible en lugar de un fallo abrupto del sistema.

7.4 Validación del Pipeline Integrado de Extremo a Extremo (*End-to-End*)

Los scripts de integración total expusieron la sinergia simbiótica entre el determinismo de la teoría de grafos y la flexibilidad cognitiva de los modelos de lenguaje.

7.4.1 Surgimiento de Alternativas Coherentes (Caso Integrado #1)

Al ingresar el texto "*Me gustaria aprender a programar en python y en javascript...*", el sistema bifurcó el análisis generando dos soluciones competitivas viables bajo el mismo umbral de restricciones (\$500, 240 horas):

- **Solución BFS (Orientada a la Inmediatez Temporal):** Generó una ruta compacta de 2 cursos (`bootcamp_python_full_speed` y `curso_introductorio_javascript`) solucionando el problema en apenas 45 horas netas, pero consumiendo \$100 del presupuesto del usuario.
- **Solución UCS/A* (Orientada a la Eficiencia Económica):** Reemplazó el bootcamp costoso por una alternativa institucional presencial de costo cero (`fundamentos_programacion_python`). Esta ruta optimizó el gasto a un mínimo histórico de \$10, a cambio de un *trade-off* temporal más demandante de 75 horas.

El componente evaluador (*Juez LLM*) demostró un alto criterio analítico al no limitarse a escupir una única respuesta. Por el contrario, estructuró un balance cualitativo de compensaciones (*trade-offs*), razonando de forma pedagógica que si el estudiante busca una introducción suave y máxima eficiencia económica debe optar por la ruta heurística, mientras que si requiere rapidez temporal debe asumir el costo de la ruta de anchura.

7.4.2 Gestión Defensiva de Restricciones Críticas (Caso Integrado #12)

El escenario definitivo de resiliencia se manifestó en el procesamiento de la cadena: "*Me gustaria saber criptografia... quiero hacerlo en solo 2 dias*". Al

mapear los 2 días a un tope estricto de 16 horas, el motor predictivo determinó que la única ruta académica real de la base de conocimiento (`criptografia_avanzada_matematica`) requería un costo temporal mínimo de 50 horas.

$$T_{\text{ofrecido}}(50h) > T_{\text{máximo}}(16h) \implies \mathcal{S}_{\text{estricta}} = \emptyset \quad (1)$$

Ante el colapso algorítmico, el planificador adaptativo forzó la viabilidad del estado relajando la frontera temporal. El análisis posterior ejecutado por el LLM justificó cognitivamente esta alteración del entorno, argumentando que la alta complejidad intrínseca de los protocolos criptográficos y la necesidad de interacción presencial directa para la asimilación matemática superan con creces el deseo inicial de inmediatez del usuario, validando la salida como una propuesta formativa valiosa y pedagógicamente consistente.

8 Limitaciones y Posibles Mejoras

A pesar de la convergencia exitosa del pipeline integrado, el análisis crítico de las pruebas de estrés expuso restricciones intrínsecas en el diseño actual del sistema, las cuales delimitan su alcance y establecen las bases para futuras líneas de investigación.

8.1 Limitaciones del Sistema Actual

- **Alineación de la Heurística de Búsqueda:** La función heurística implementada, $h(n) = |G \setminus \text{effects}(S_n)|$, asume una independencia lineal entre las metas. Esto provoca que en árboles lógicos con alta densidad de prerequisites concatenados (como en los escenarios avanzados de Inteligencia Artificial), la heurística subestime la distancia real al estado objetivo, incrementando marginalmente la tasa de nodos expandidos.
- **Dependencia del Contexto del Catálogo:** El extractor semántico `LLMInterface` depende estrictamente de un vocabulario cerrado de 30 habilidades. Ante consultas ambiguas o con jerga industrial no indexada, el modelo se ve obligado a inyectar valores centinelas (10^{13}) o vectores vacíos, limitando el dinamismo adaptativo ante ofertas académicas emergentes.
- **Relajación Unidimensional en el Planificador:** El *Adaptive Planner* ejecuta actualmente una relajación en cascada sobre umbrales rígidos. Al enfrentarse a restricciones humanas complejas, prioriza el cumplimiento del objetivo educativo relajando el tiempo o el dinero de forma aislada, sin ponderar funciones de utilidad personalizadas para el usuario.

8.2 Líneas de Trabajo Futuro y Mejoras Propuestas

- **Optimización de la Heurística mediante Grafos de Planificación:** Se propone transicionar hacia heurísticas basadas en la distensión de grafos

(como *Fast-Forward* o *Graphplan*), capaces de evaluar de forma simultánea las interdependencias y efectos mutuos de los cursos para guiar a A^* con mayor precisión.

- **Ecosistema de Embeddings Dinámicos:** Reemplazar el mapeo sintáctico estático por un espacio vectorial continuo utilizando text embeddings. Esto permitiría computar la similitud del coseno entre los deseos del usuario y los objetivos pedagógicos de las asignaturas, eliminando la necesidad de un catálogo rígido de habilidades.
- **Algoritmos Genéticos y Aprendizaje por Refuerzo para el Juez LLM:** Implementar algoritmos evolutivos multiobjetivo (como NSGA-II) para generar un frente de Pareto de trayectorias óptimas. Esto le proporcionaría al "Juez LLM" un abanico de alternativas cuantitativamente diversas (balanceando costo, tiempo y fatiga cognitiva) para su posterior curaduría cualitativa.